Eigenes CSS

Termin 3 · 13.10.2026

FÄLLIG
bis Termin 4 am 20.10.

ZEITBEDARF
ca. 2 Stunden

BAUSTEINE IM REPO
t3-kaskade-und-spezifitaet, t3-box-modell, t3-sticky-header-badge

KONTAKT
dhbw@theaisoftwarecompany.com

Diese Woche bekommt euer Projekt ein Gesicht – und ihr bekommt das Vokabular, mit dem man einer KI sagt, was sie ändern soll. „Mach es schöner" ist kein Auftrag. „nav als Flex-Container, sticky mit top 0" schon.

Teil A · Pflicht

A1 Ein Stylesheet für alles

Legt `style.css` an und bindet es auf **allen** Seiten ein. Ganz oben:

```
* { box-sizing: border-box; }

:root {
  --bg: #ffffff;
  --text: #1f1f1f;
  --akzent: #0000ff;
  --abstand: 1rem;
  --radius: 12px;
}
```

Benutzt die Variablen danach überall statt roher Werte. Maximal **drei** Schriftgrößen und **eine** Akzentfarbe im ganzen Projekt.

→ **Fertig, wenn** ihr die Akzentfarbe an **einer** Stelle ändern könnt und sich die ganze Seite mitändert.

A2 Navigation gestalten

Die `nav` bekommt Abstände, Farben und zwei Zustände:

- `:hover` für die Maus
- `:focus-visible` für die Tastatur – den Fokusrahmen **nicht** entfernen, sondern gestalten

→ **Fertig, wenn** man beim Durchtabben jederzeit sieht, wo man ist.

A3 Box-Modell in eurem Layout

Baut eure Inhalte in Karten oder Abschnitte mit `padding`, `border-radius` und Abständen über `gap` oder `margin`. Begrenzt die Textbreite:

```
main { max-width: 65ch; margin-inline: auto; }
```

→ **Fertig, wenn** bei 360 px Fensterbreite nichts seitlich herausragt und die Seite nicht horizontal scrollt.

A4 Sticky Header und ein Badge

Header klebt oben (`position: sticky; top: 0; z-index: 10`). Irgendwo im Projekt sitzt ein kleines Element in der Ecke seines Containers – das Paar `relative` am Elternteil, `absolute` am Kind.

→ **Fertig, wenn** beides beim Scrollen und bei schmalem Fenster hält.

A5 Eine Kollision nachweisen

Sucht in eurem eigenen CSS eine Stelle, an der **zwei Regeln dasselbe Element treffen**. Findet in den DevTools die durchgestrichene Regel.

Schreibt drei Zeilen in die README: welche Regeln, welche gewinnt, warum.

→ **Fertig, wenn** die Begründung die Spezifität nennt und nicht „weil sie weiter unten steht" – außer das ist wirklich der Grund.

Teil B · Bonus (+1 Punkt)

Dark Mode über `prefers-color-scheme`. Nur die Farbwerte in `:root` tauschen, keine zweite Kopie des Stylesheets.

Prüfen: DevTools → Befehlspalette (`Cmd/Strg + Shift + P`) → „Emulate CSS prefers-color-scheme: dark".

Teil C · Verständnisfragen

1. Zwei Regeln treffen denselben Link: `nav a { color: grey }` und `.aktiv { color: blue }`. Welche gewinnt, und warum?
2. Warum `rem` für Schriftgrößen und nicht `px`? Für wen macht das einen Unterschied?
3. Ihr setzt `z-index: 999` und es passiert nichts. Woran liegt das vermutlich?

So arbeitest du hier mit KI

Der Unterschied dieser Woche in einem Bild:

> *„Mach die Navigation schöner und dass sie oben bleibt."*

gegen

> *„Mach die `nav` zu einem Flex-Container, Abstand zwischen den Links 24 px, `position: sticky` mit `top: 0` und `z-index` über dem Inhalt."*

Links bekommt ihr irgendetwas. Rechts bekommt ihr, was ihr wolltet – und könnt im Code nachsehen, ob es stimmt.

Gut für diese Woche:

> *„Hier sind zwei CSS-Regeln, die dasselbe Element treffen. Erklär mir, welche gewinnt und wie sich die Spezifität berechnet. Zeig mir die Rechnung, nicht nur das Ergebnis."*